Python Library: logging

Source: https://docs.python.org/3.12/library/logging.html

License: Python Software Foundation License Version 2
(https://docs.python.org/3.12/license.html)

Copyright © 2001 Python Software Foundation; All Rights Reserved.

Adaptation note: HTML markup and navigation were removed, whitespace was normalized, and selected pages were converted to plain text, PDF, or DOCX. No technical claims were intentionally changed.

logging · Logging facility for Python · Python 3.12.16 documentation

logging · Logging facility for Python

Source code: Lib/logging/__init__.py

Important

This page contains the API reference information. For tutorial

information and discussion of more advanced topics, see

Basic Tutorial

Advanced Tutorial

Logging Cookbook

This module defines functions and classes which implement a flexible event

logging system for applications and libraries.

The key benefit of having the logging API provided by a standard library module

is that all Python modules can participate in logging, so your application log

can include your own messages integrated with messages from third-party

modules.

Here·s a simple example of idiomatic usage:

```
# myapp.py

import logging

import mylib

logger = logging.getLogger(__name__)

def main():

logging.basicConfig(filename='myapp.log', level=logging.INFO)

logger.info('Started')

mylib.do_something()

logger.info('Finished')

if __name__ == '__main__':

main()

# mylib.py

import logging

logger = logging.getLogger(__name__)

def do_something():

logger.info('Doing something')
```

If you run myapp.py, you should see this in myapp.log:

```
INFO:__main__:Started

INFO:mylib:Doing something

INFO:__main__:Finished
```

The key feature of this idiomatic usage is that the majority of code is simply

creating a module level logger with getLogger(__name__), and using that

logger to do any needed logging. This is concise, while allowing downstream code fine-grained control if needed. Logged messages to the module-level logger get forwarded to handlers of loggers in higher-level modules, all the way up to the highest-level logger known as the root logger; this approach is known as hierarchical logging.

For logging to be useful, it needs to be configured: setting the levels and destinations for each logger, potentially changing how specific modules log, often based on command-line arguments or application configuration. In most cases, like the one above, only the root logger needs to be so configured, since all the lower level loggers at module level eventually forward their messages to its handlers. basicConfig() provides a quick way to configure the root logger that handles many use cases.

The module provides a lot of functionality and flexibility. If you are unfamiliar with logging, the best way to get to grips with it is to view the tutorials (see the links above and on the right).

The basic classes defined by the module, together with their attributes and methods, are listed in the sections below.

Loggers expose the interface that application code directly uses.

Handlers send the log records (created by loggers) to the appropriate destination.

Filters provide a finer grained facility for determining which log records to output.

Formatters specify the layout of log records in the final output.

Logger Objects

Loggers have the following attributes and methods. Note that Loggers should

NEVER be instantiated directly, but always through the module-level function

logging.getLogger(name). Multiple calls to getLogger() with the same

name will always return a reference to the same Logger object.

The name is potentially a period-separated hierarchical value, like

foo.bar.baz (though it could also be just plain foo, for example).

Loggers that are further down in the hierarchical list are children of loggers

higher up in the list. For example, given a logger with a name of foo,

loggers with names of foo.bar, foo.bar.baz, and foo.bam are all

descendants of foo. In addition, all loggers are descendants of the root

logger. The logger name hierarchy is analogous to the Python package hierarchy,

and identical to it if you organise your loggers on a per-module basis using

the recommended construction logging.getLogger(__name__). That·s because

in a module, __name__ is the module·s name in the Python package namespace.

class logging.Logger

name

This is the logger·s name, and is the value that was passed to getLogger()

to obtain the logger.

Note

This attribute should be treated as read-only.

level

The threshold of this logger, as set by the setLevel() method.

Note

Do not set this attribute directly - always use setLevel(),

which has checks for the level passed to it.

parent

The parent logger of this logger. It may change based on later instantiation

of loggers which are higher up in the namespace hierarchy.

Note

This value should be treated as read-only.

propagate

If this attribute evaluates to true, events logged to this logger will be

passed to the handlers of higher level (ancestor) loggers, in addition to

any handlers attached to this logger. Messages are passed directly to the

ancestor loggers· handlers - neither the level nor filters of the ancestor

loggers in question are considered.

If this evaluates to false, logging messages are not passed to the handlers

of ancestor loggers.

Spelling it out with an example: If the propagate attribute of the logger named

A.B.C evaluates to true, any event logged to A.B.C via a method call such as

logging.getLogger('A.B.C').error(...) will [subject to passing that logger·s

level and filter settings] be passed in turn to any handlers attached to loggers

named A.B, A and the root logger, after first being passed to any handlers attached to A.B.C. If any logger in the chain A.B.C, A.B, A has its propagate attribute set to false, then that is the last logger whose handlers are offered the event to handle, and propagation stops at that point.

The constructor sets this attribute to True.

Note

If you attach a handler to a logger and one or more of its ancestors, it may emit the same record multiple times. In general, you should not need to attach a handler to more than one logger - if you just attach it to the appropriate logger which is highest in the logger hierarchy, then it will see all events logged by all descendant loggers, provided that their propagate setting is left set to True. A common scenario is to attach handlers only to the root logger, and to let propagation take care of the rest.

handlers

The list of handlers directly attached to this logger instance.

Note

This attribute should be treated as read-only; it is normally changed via the addHandler() and removeHandler() methods, which use locks to ensure thread-safe operation.

disabled

This attribute disables handling of any events. It is set to False in the

initializer, and only changed by logging configuration code.

Note

This attribute should be treated as read-only.

setLevel(level)

Sets the threshold for this logger to level. Logging messages which are less

severe than level will be ignored; logging messages which have severity level

or higher will be emitted by whichever handler or handlers service this logger,

unless a handler·s level has been set to a higher severity level than level.

When a logger is created, the level is set to NOTSET (which causes

all messages to be processed when the logger is the root logger, or delegation

to the parent when the logger is a non-root logger). Note that the root logger

is created with level WARNING.

The term ·delegation to the parent· means that if a logger has a level of

NOTSET, its chain of ancestor loggers is traversed until either an ancestor with

a level other than NOTSET is found, or the root is reached.

If an ancestor is found with a level other than NOTSET, then that ancestor·s

level is treated as the effective level of the logger where the ancestor search

began, and is used to determine how a logging event is handled.

If the root is reached, and it has a level of NOTSET, then all messages will be

processed. Otherwise, the root·s level will be used as the effective level.

See Logging Levels for a list of levels.

Changed in version 3.2: The level parameter now accepts a string representation of the

level such as ·INFO· as an alternative to the integer constants

such as INFO. Note, however, that levels are internally stored

as integers, and methods such as e.g. getEffectiveLevel() and

isEnabledFor() will return/expect to be passed integers.

isEnabledFor(level)

Indicates if a message of severity level would be processed by this logger.

This method checks first the module-level level set by

logging.disable(level) and then the logger·s effective level as determined

by getEffectiveLevel().

getEffectiveLevel()

Indicates the effective level for this logger. If a value other than

NOTSET has been set using setLevel(), it is returned. Otherwise,

the hierarchy is traversed towards the root until a value other than

NOTSET is found, and that value is returned. The value returned is

an integer, typically one of logging.DEBUG, logging.INFO

etc.

getChild(suffix)

Returns a logger which is a descendant to this logger, as determined by the suffix.

Thus, logging.getLogger('abc').getChild('def.ghi') would return the same

logger as would be returned by logging.getLogger('abc.def.ghi'). This is a

convenience method, useful when the parent logger is named using e.g. __name__

rather than a literal string.

Added in version 3.2.

getChildren()

Returns a set of loggers which are immediate children of this logger. So for

example logging.getLogger().getChildren() might return a set containing

loggers named foo and bar, but a logger named foo.bar wouldn·t be

included in the set. Likewise, logging.getLogger('foo').getChildren() might

return a set including a logger named foo.bar, but it wouldn·t include one

named foo.bar.baz.

Added in version 3.12.

debug(msg, *args, **kwargs)

Logs a message with level DEBUG on this logger. The msg is the

message format string, and the args are the arguments which are merged into

msg using the string formatting operator. (Note that this means that you can

use keywords in the format string, together with a single dictionary argument.)

No % formatting operation is performed on msg when no args are supplied.

There are four keyword arguments in kwargs which are inspected:

exc_info, stack_info, stacklevel and extra.

If exc_info does not evaluate as false, it causes exception information to be

added to the logging message. If an exception tuple (in the format returned by

sys.exc_info()) or an exception instance is provided, it is used;

otherwise, sys.exc_info() is called to get the exception information.

The second optional keyword argument is stack_info, which defaults to

False. If true, stack information is added to the logging

message, including the actual logging call. Note that this is not the same

stack information as that displayed through specifying exc_info: The

former is stack frames from the bottom of the stack up to the logging call

in the current thread, whereas the latter is information about stack frames

which have been unwound, following an exception, while searching for

exception handlers.

You can specify stack_info independently of exc_info, e.g. to just show

how you got to a certain point in your code, even when no exceptions were

raised. The stack frames are printed following a header line which says:

Stack (most recent call last):

This mimics the Traceback (most recent call last): which is used when

displaying exception frames.

The third optional keyword argument is stacklevel, which defaults to 1.

If greater than 1, the corresponding number of stack frames are skipped

when computing the line number and function name set in the LogRecord

created for the logging event. This can be used in logging helpers so that

the function name, filename and line number recorded are not the information

for the helper function/method, but rather its caller. The name of this

parameter mirrors the equivalent one in the warnings module.

The fourth keyword argument is extra which can be used to pass a

dictionary which is used to populate the __dict__ of the

LogRecord

created for the logging event with user-defined attributes. These custom

attributes can then be used as you like. For example, they could be

incorporated into logged messages. For example:

FORMAT = '%(asctime)s %(clientip)-15s %(user)-8s %(message)s'

logging.basicConfig(format=FORMAT)

d = {'clientip': '192.168.0.1', 'user': 'fbloggs'}

logger = logging.getLogger('tcpserver')

logger.warning('Protocol problem: %s', 'connection reset', extra=d)

would print something like

2006-02-08 22:20:02,165 192.168.0.1 fbloggs Protocol problem: connection reset

The keys in the dictionary passed in extra should not clash with the keys used

by the logging system. (See the section on LogRecord attributes for more

information on which keys are used by the logging system.)

If you choose to use these attributes in logged messages, you need to exercise

some care. In the above example, for instance, the Formatter has been

set up with a format string which expects ·clientip· and ·user· in the attribute

dictionary of the LogRecord. If these are missing, the message will

not be logged because a string formatting exception will occur. So in this case,

you always need to pass the extra dictionary with these keys.

While this might be annoying, this feature is intended for use in specialized

circumstances, such as multi-threaded servers where the same code executes in

many contexts, and interesting conditions which arise are dependent on this

context (such as remote client IP address and authenticated user name, in the

above example). In such circumstances, it is likely that specialized

Formatters would be used with particular Handlers.

If no handler is attached to this logger (or any of its ancestors,

taking into account the relevant Logger.propagate attributes),

the message will be sent to the handler set on lastResort.

Changed in version 3.2: The stack_info parameter was added.

Changed in version 3.5: The exc_info parameter can now accept exception instances.

Changed in version 3.8: The stacklevel parameter was added.

info(msg, *args, **kwargs)

Logs a message with level INFO on this logger. The arguments are

interpreted as for debug().

warning(msg, *args, **kwargs)

Logs a message with level WARNING on this logger. The arguments are

interpreted as for debug().

Note

There is an obsolete method warn which is functionally

identical to warning. As warn is deprecated, please do not use

it - use warning instead.

error(msg, *args, **kwargs)

Logs a message with level ERROR on this logger. The arguments are

interpreted as for debug().

critical(msg, *args, **kwargs)

Logs a message with level CRITICAL on this logger. The arguments are

interpreted as for debug().

log(level, msg, *args, **kwargs)

Logs a message with integer level level on this logger. The other arguments are

interpreted as for debug().

exception(msg, *args, **kwargs)

Logs a message with level ERROR on this logger. The arguments are

interpreted as for debug(). Exception info is added to the logging

message. This method should only be called from an exception handler.

addFilter(filter)

Adds the specified filter filter to this logger.

removeFilter(filter)

Removes the specified filter filter from this logger.

filter(record)

Apply this logger·s filters to the record and return True if the

record is to be processed. The filters are consulted in turn, until one of

them returns a false value. If none of them return a false value, the record

will be processed (passed to handlers). If one returns a false value, no

further processing of the record occurs.

addHandler(hdlr)

Adds the specified handler hdlr to this logger.

removeHandler(hdlr)

Removes the specified handler hdlr from this logger.

findCaller(stack_info=False, stacklevel=1)

Finds the caller·s source filename and line number. Returns the filename, line

number, function name and stack information as a 4-element tuple. The stack

information is returned as None unless stack_info is True.

The stacklevel parameter is passed from code calling the debug()

and other APIs. If greater than 1, the excess is used to skip stack frames

before determining the values to be returned. This will generally be useful

when calling logging APIs from helper/wrapper code, so that the information

in the event log refers not to the helper/wrapper code, but to the code that

calls it.

handle(record)

Handles a record by passing it to all handlers associated with this logger and

its ancestors (until a false value of propagate is found). This method is used

for unpickled records received from a socket, as well as those created locally.

Logger-level filtering is applied using filter().

makeRecord(name, level, fn, lno, msg, args, exc_info, func=None, extra=None, sinfo=None)

This is a factory method which can be overridden in subclasses to create

specialized LogRecord instances.

hasHandlers()

Checks to see if this logger has any handlers configured. This is done by

looking for handlers in this logger and its parents in the logger hierarchy.

Returns True if a handler was found, else False. The method stops searching

up the hierarchy whenever a logger with the ·propagate· attribute set to

false is found - that will be the last logger which is checked for the

existence of handlers.

Added in version 3.2.

Changed in version 3.7: Loggers can now be pickled and unpickled.

Logging Levels

The numeric values of logging levels are given in the following table. These are

primarily of interest if you want to define your own levels, and need them to

have specific values relative to the predefined levels. If you define a level

with the same numeric value, it overwrites the predefined value; the predefined

name is lost.

Level

Numeric value

What it means / When to use it

logging.NOTSET

0

When set on a logger, indicates that

ancestor loggers are to be consulted

to determine the effective level.

If that still resolves to

NOTSET, then all events

are logged. When set on a handler,

all events are handled.

logging.DEBUG

10

Detailed information, typically only

of interest to a developer trying to

diagnose a problem.

logging.INFO

20

Confirmation that things are working

as expected.

logging.WARNING

30

An indication that something

unexpected happened, or that a

problem might occur in the near

future (e.g. ·disk space low·). The

software is still working as

expected.

logging.ERROR

40

Due to a more serious problem, the

software has not been able to

perform some function.

logging.CRITICAL

50

A serious error, indicating that the

program itself may be unable to

continue running.

Handler Objects

Handlers have the following attributes and methods. Note that Handler

is never instantiated directly; this class acts as a base for more useful

subclasses. However, the __init__() method in subclasses needs to call

Handler.__init__().

class logging.Handler

__init__(level=NOTSET)

Initializes the Handler instance by setting its level, setting the list

of filters to the empty list and creating a lock (using createLock()) for

serializing access to an I/O mechanism.

createLock()

Initializes a thread lock which can be used to serialize access to underlying

I/O functionality which may not be threadsafe.

acquire()

Acquires the thread lock created with createLock().

release()

Releases the thread lock acquired with acquire().

setLevel(level)

Sets the threshold for this handler to level. Logging messages which are

less severe than level will be ignored. When a handler is created, the

level is set to NOTSET (which causes all messages to be

processed).

See Logging Levels for a list of levels.

Changed in version 3.2: The level parameter now accepts a string representation of the

level such as ·INFO· as an alternative to the integer constants

such as INFO.

setFormatter(fmt)

Sets the formatter for this handler to fmt.

The fmt argument must be a Formatter instance or None.

addFilter(filter)

Adds the specified filter filter to this handler.

removeFilter(filter)

Removes the specified filter filter from this handler.

filter(record)

Apply this handler·s filters to the record and return True if the

record is to be processed. The filters are consulted in turn, until one of

them returns a false value. If none of them return a false value, the record

will be emitted. If one returns a false value, the handler will not emit the

record.

flush()

Ensure all logging output has been flushed. This version does nothing and is

intended to be implemented by subclasses.

close()

Tidy up any resources used by the handler. This version does no output

but removes the handler from an internal map of handlers, which is used

for handler lookup by name.

Subclasses should ensure that this gets called from overridden close()

methods.

handle(record)

Conditionally emits the specified logging record, depending on filters which may

have been added to the handler. Wraps the actual emission of the record with

acquisition/release of the I/O thread lock.

handleError(record)

This method should be called from handlers when an exception is encountered

during an emit() call. If the module-level attribute

raiseExceptions is False, exceptions get silently ignored. This is

what is mostly wanted for a logging system - most users will not care about

errors in the logging system, they are more interested in application

errors. You could, however, replace this with a custom handler if you wish.

The specified record is the one which was being processed when the exception

occurred. (The default value of raiseExceptions is True, as that is

more useful during development).

format(record)

Do formatting for a record - if a formatter is set, use it. Otherwise, use the

default formatter for the module.

emit(record)

Do whatever it takes to actually log the specified logging record. This version

is intended to be implemented by subclasses and so raises a

NotImplementedError.

Warning

This method is called after a handler-level lock is acquired, which

is released after this method returns. When you override this method, note

that you should be careful when calling anything that invokes other parts of

the logging API which might do locking, because that might result in a

deadlock. Specifically:

Logging configuration APIs acquire the module-level lock, and then

individual handler-level locks as those handlers are configured.

Many logging APIs lock the module-level lock. If such an API is called

from this method, it could cause a deadlock if a configuration call is

made on another thread, because that thread will try to acquire the

module-level lock before the handler-level lock, whereas this thread

tries to acquire the module-level lock after the handler-level lock

(because in this method, the handler-level lock has already been acquired).

For a list of handlers included as standard, see logging.handlers.

Formatter Objects

class logging.Formatter(fmt=None, datefmt=None, style='%', validate=True, *, defaults=None)

Responsible for converting a LogRecord to an output string

to be interpreted by a human or external system.

Parameters:

fmt (str) · A format string in the given style for

the logged output as a whole.

The possible mapping keys are drawn from the LogRecord object·s

LogRecord attributes.

If not specified, '%(message)s' is used,

which is just the logged message.

datefmt (str) · A format string in the given style for

the date/time portion of the logged output.

If not specified, the default described in formatTime() is used.

style (str) · Can be one of '%', '{' or '$' and determines

how the format string will be merged with its data: using one of

printf-style String Formatting (%), str.format() ({)

or string.Template ($). This only applies to

fmt and datefmt (e.g. '%(message)s' versus '{message}'),

not to the actual log messages passed to the logging methods.

However, there are other ways

to use {- and $-formatting for log messages.

validate (bool) · If True (the default), incorrect or mismatched

fmt and style will raise a ValueError; for example,

logging.Formatter('%(asctime)s - %(message)s', style='{').

defaults (dict[str, Any]) · A dictionary with default values to use in custom fields.

For example,

logging.Formatter('%(ip)s %(message)s', defaults={"ip": None})

Changed in version 3.2: Added the style parameter.

Changed in version 3.8: Added the validate parameter.

Changed in version 3.10: Added the defaults parameter.

format(record)

The record·s attribute dictionary is used as the operand to a string

formatting operation. Returns the resulting string. Before formatting the

dictionary, a couple of preparatory steps are carried out. The message

attribute of the record is computed using msg % args. If the

formatting string contains '(asctime)', formatTime() is called

to format the event time. If there is exception information, it is

formatted using formatException() and appended to the message. Note

that the formatted exception information is cached in attribute

exc_text. This is useful because the exception information can be

pickled and sent across the wire, but you should be careful if you have

more than one Formatter subclass which customizes the formatting

of exception information. In this case, you will have to clear the cached

value (by setting the exc_text attribute to None) after a formatter

has done its formatting, so that the next formatter to handle the event

doesn·t use the cached value, but recalculates it afresh.

If stack information is available, it·s appended after the exception

information, using formatStack() to transform it if necessary.

formatTime(record, datefmt=None)

This method should be called from format() by a formatter which

wants to make use of a formatted time. This method can be overridden in

formatters to provide for any specific requirement, but the basic behavior

is as follows: if datefmt (a string) is specified, it is used with

time.strftime() to format the creation time of the

record. Otherwise, the format ·%Y-%m-%d %H:%M:%S,uuu· is used, where the

uuu part is a millisecond value and the other letters are as per the

time.strftime() documentation. An example time in this format is

2003-01-23 00:29:50,411. The resulting string is returned.

This function uses a user-configurable function to convert the creation

time to a tuple. By default, time.localtime() is used; to change

this for a particular formatter instance, set the converter attribute

to a function with the same signature as time.localtime() or

time.gmtime(). To change it for all formatters, for example if you

want all logging times to be shown in GMT, set the converter

attribute in the Formatter class.

Changed in version 3.3: Previously, the default format was hard-coded as in this example:

2010-09-06 22:38:15,292 where the part before the comma is

handled by a strptime format string ('%Y-%m-%d %H:%M:%S'), and the

part after the comma is a millisecond value. Because strptime does not

have a format placeholder for milliseconds, the millisecond value is

appended using another format string, '%s,%03d' · and both of these

format strings have been hardcoded into this method. With the change,

these strings are defined as class-level attributes which can be

overridden at the instance level when desired. The names of the

attributes are default_time_format (for the strptime format string)

and default_msec_format (for appending the millisecond value).

Changed in version 3.9: The default_msec_format can be None.

formatException(exc_info)

Formats the specified exception information (a standard exception tuple as

returned by sys.exc_info()) as a string. This default implementation

just uses traceback.print_exception(). The resulting string is

returned.

formatStack(stack_info)

Formats the specified stack information (a string as returned by

traceback.print_stack(), but with the last newline removed) as a

string. This default implementation just returns the input value.

class logging.BufferingFormatter(linefmt=None)

A base formatter class suitable for subclassing when you want to format a

number of records. You can pass a Formatter instance which you want

to use to format each line (that corresponds to a single record). If not

specified, the default formatter (which just outputs the event message) is

used as the line formatter.

formatHeader(records)

Return a header for a list of records. The base implementation just

returns the empty string. You will need to override this method if you

want specific behaviour, e.g. to show the count of records, a title or a

separator line.

formatFooter(records)

Return a footer for a list of records. The base implementation just

returns the empty string. You will need to override this method if you

want specific behaviour, e.g. to show the count of records or a separator

line.

format(records)

Return formatted text for a list of records. The base implementation

just returns the empty string if there are no records; otherwise, it

returns the concatenation of the header, each record formatted with the

line formatter, and the footer.

Filter Objects

Filters can be used by Handlers and Loggers for more sophisticated

filtering than is provided by levels. The base filter class only allows events

which are below a certain point in the logger hierarchy. For example, a filter

initialized with ·A.B· will allow events logged by loggers ·A.B·, ·A.B.C·,

·A.B.C.D·, ·A.B.D· etc. but not ·A.BB·, ·B.A.B· etc. If initialized with the

empty string, all events are passed.

class logging.Filter(name='')

Returns an instance of the Filter class. If name is specified, it

names a logger which, together with its children, will have its events allowed

through the filter. If name is the empty string, allows every event.

filter(record)

Is the specified record to be logged? Returns false for no, true for

yes. Filters can either modify log records in-place or return a completely

different record instance which will replace the original

log record in any future processing of the event.

Note that filters attached to handlers are consulted before an event is

emitted by the handler, whereas filters attached to loggers are consulted

whenever an event is logged (using debug(), info(),

etc.), before sending an event to handlers. This means that events which have

been generated by descendant loggers will not be filtered by a logger·s filter

setting, unless the filter has also been applied to those descendant loggers.

You don·t actually need to subclass Filter: you can pass any instance

which has a filter method with the same semantics.

Changed in version 3.2: You don·t need to create specialized Filter classes, or use other

classes with a filter method: you can use a function (or other

callable) as a filter. The filtering logic will check to see if the filter

object has a filter attribute: if it does, it·s assumed to be a

Filter and its filter() method is called. Otherwise, it·s

assumed to be a callable and called with the record as the single

parameter. The returned value should conform to that returned by

filter().

Changed in version 3.12: You can now return a LogRecord instance from filters to replace

the log record rather than modifying it in place. This allows filters attached to

a Handler to modify the log record before it is emitted, without

having side effects on other handlers.

Although filters are used primarily to filter records based on more

sophisticated criteria than levels, they get to see every record which is

processed by the handler or logger they·re attached to: this can be useful if

you want to do things like counting how many records were processed by a

particular logger or handler, or adding, changing or removing attributes in

the LogRecord being processed. Obviously changing the LogRecord needs

to be done with some care, but it does allow the injection of contextual

information into logs (see Using Filters to impart contextual information).

LogRecord Objects

LogRecord instances are created automatically by the Logger

every time something is logged, and can be created manually via

makeLogRecord() (for example, from a pickled event received over the

wire).

class logging.LogRecord(name, level, pathname, lineno, msg, args, exc_info, func=None, sinfo=None)

Contains all the information pertinent to the event being logged.

The primary information is passed in msg and args,

which are combined using msg % args to create

the message attribute of the record.

Parameters:

name (str) · The name of the logger used to log the event

represented by this LogRecord.

Note that the logger name in the LogRecord

will always have this value,

even though it may be emitted by a handler

attached to a different (ancestor) logger.

level (int) · The numeric level of the logging event

(such as 10 for DEBUG, 20 for INFO, etc).

Note that this is converted to two attributes of the LogRecord:

levelno for the numeric value

and levelname for the corresponding level name.

pathname (str) · The full string path of the source file

where the logging call was made.

lineno (int) · The line number in the source file

where the logging call was made.

msg (Any) · The event description message,

which can be a %-format string with placeholders for variable data,

or an arbitrary object (see Using arbitrary objects as messages).

args (tuple | dict[str, Any]) · Variable data to merge into the msg argument

to obtain the event description.

exc_info (tuple[type[BaseException], BaseException, types.TracebackType] | None) · An exception tuple with the current exception information,

as returned by sys.exc_info(),

or None if no exception information is available.

func (str | None) · The name of the function or method

from which the logging call was invoked.

sinfo (str | None) · A text string representing stack information

from the base of the stack in the current thread,

up to the logging call.

getMessage()

Returns the message for this LogRecord instance after merging any

user-supplied arguments with the message. If the user-supplied message

argument to the logging call is not a string, str() is called on it to

convert it to a string. This allows use of user-defined classes as

messages, whose __str__ method can return the actual format string to

be used.

Changed in version 3.2: The creation of a LogRecord has been made more configurable by

providing a factory which is used to create the record. The factory can be

set using getLogRecordFactory() and setLogRecordFactory()

(see this for the factory·s signature).

This functionality can be used to inject your own values into a

LogRecord at creation time. You can use the following pattern:

old_factory = logging.getLogRecordFactory()

def record_factory(*args, **kwargs):

record = old_factory(*args, **kwargs)

record.custom_attribute = 0xdecafbad

return record

logging.setLogRecordFactory(record_factory)

With this pattern, multiple factories could be chained, and as long

as they don·t overwrite each other·s attributes or unintentionally

overwrite the standard attributes listed above, there should be no surprises.

LogRecord attributes

The LogRecord has a number of attributes, most of which are derived from the parameters to the constructor. (Note that the names do not always correspond exactly between the LogRecord constructor parameters and the LogRecord attributes.) These attributes can be used to merge data from the record into the format string. The following table lists (in alphabetical order) the attribute names, their meanings and the corresponding placeholder in a %-style format string.

If you are using {}-formatting (str.format()), you can use {attrname} as the placeholder in the format string. If you are using $-formatting (string.Template), use the form ${attrname}. In both cases, of course, replace attrname with the actual attribute name you want to use.

In the case of {}-formatting, you can specify formatting flags by placing them after the attribute name, separated from it with a colon. For example: a placeholder of {msecs:03.0f} would format a millisecond value of 4 as 004. Refer to the str.format() documentation for full details on the options available to you.

Attribute name

Format

Description

args

You shouldn·t need to

format this yourself.

The tuple of arguments merged into msg to

produce message, or a dict whose values

are used for the merge (when there is only one

argument, and it is a dictionary).

asctime

%(asctime)s

Human-readable time when the

LogRecord was created. By default

this is of the form ·2003-07-08 16:49:45,896·

(the numbers after the comma are millisecond

portion of the time).

created

%(created)f

Time when the LogRecord was created

(as returned by time.time()).

exc_info

You shouldn·t need to

format this yourself.

Exception tuple (à la sys.exc_info) or,

if no exception has occurred, None.

filename

%(filename)s

Filename portion of pathname.

funcName

%(funcName)s

Name of function containing the logging call.

levelname

%(levelname)s

Text logging level for the message

('DEBUG', 'INFO', 'WARNING',

'ERROR', 'CRITICAL').

levelno

%(levelno)s

Numeric logging level for the message

(DEBUG, INFO,

WARNING, ERROR,

CRITICAL).

lineno

%(lineno)d

Source line number where the logging call was

issued (if available).

message

%(message)s

The logged message, computed as msg %

args. This is set when

Formatter.format() is invoked.

module

%(module)s

Module (name portion of filename).

msecs

%(msecs)d

Millisecond portion of the time when the

LogRecord was created.

msg

You shouldn·t need to

format this yourself.

The format string passed in the original

logging call. Merged with args to

produce message, or an arbitrary object

(see Using arbitrary objects as messages).

name

%(name)s

Name of the logger used to log the call.

pathname

%(pathname)s

Full pathname of the source file where the

logging call was issued (if available).

process

%(process)d

Process ID (if available).

processName

%(processName)s

Process name (if available).

relativeCreated

%(relativeCreated)d

Time in milliseconds when the LogRecord was

created, relative to the time the logging

module was loaded.

stack_info

You shouldn·t need to

format this yourself.

Stack frame information (where available)

from the bottom of the stack in the current

thread, up to and including the stack frame

of the logging call which resulted in the

creation of this record.

thread

%(thread)d

Thread ID (if available).

threadName

%(threadName)s

Thread name (if available).

taskName

%(taskName)s

asyncio.Task name (if available).

Changed in version 3.1: processName was added.

Changed in version 3.12: taskName was added.

LoggerAdapter Objects

LoggerAdapter instances are used to conveniently pass contextual

information into logging calls. For a usage example, see the section on

adding contextual information to your logging output.

class logging.LoggerAdapter(logger, extra)

Returns an instance of LoggerAdapter initialized with an

underlying Logger instance and a dict-like object.

process(msg, kwargs)

Modifies the message and/or keyword arguments passed to a logging call in

order to insert contextual information. This implementation takes the object

passed as extra to the constructor and adds it to kwargs using key

·extra·. The return value is a (msg, kwargs) tuple which has the

(possibly modified) versions of the arguments passed in.

manager

Delegates to the underlying manager on logger.

_log

Delegates to the underlying _log() method on logger.

In addition to the above, LoggerAdapter supports the following

methods of Logger: debug(), info(),

warning(), error(), exception(),

critical(), log(), isEnabledFor(),

getEffectiveLevel(), setLevel() and

hasHandlers(). These methods have the same signatures as their

counterparts in Logger, so you can use the two types of instances

interchangeably.

Changed in version 3.2: The isEnabledFor(), getEffectiveLevel(),

setLevel() and hasHandlers() methods were added

to LoggerAdapter. These methods delegate to the underlying logger.

Changed in version 3.6: Attribute manager and method _log() were added, which

delegate to the underlying logger and allow adapters to be nested.

Thread Safety

The logging module is intended to be thread-safe without any special work

needing to be done by its clients. It achieves this though using threading

locks; there is one lock to serialize access to the module·s shared data, and

each handler also creates a lock to serialize access to its underlying I/O.

If you are implementing asynchronous signal handlers using the signal

module, you may not be able to use logging from within such handlers. This is

because lock implementations in the threading module are not always

re-entrant, and so cannot be invoked from such signal handlers.

Module-Level Functions

In addition to the classes described above, there are a number of module-level

functions.

logging.getLogger(name=None)

Return a logger with the specified name or, if name is None, return the

root logger of the hierarchy. If specified, the name is typically a

dot-separated hierarchical name like ·a·, ·a.b· or ·a.b.c.d·. Choice

of these names is entirely up to the developer who is using logging, though

it is recommended that __name__ be used unless you have a specific

reason for not doing that, as mentioned in Logger Objects.

All calls to this function with a given name return the same logger instance.

This means that logger instances never need to be passed between different parts

of an application.

logging.getLoggerClass()

Return either the standard Logger class, or the last class passed to

setLoggerClass(). This function may be called from within a new class

definition, to ensure that installing a customized Logger class will

not undo customizations already applied by other code. For example:

class MyLogger(logging.getLoggerClass()):

... override behaviour here

logging.getLogRecordFactory()

Return a callable which is used to create a LogRecord.

Added in version 3.2: This function has been provided, along with setLogRecordFactory(),

to allow developers more control over how the LogRecord

representing a logging event is constructed.

See setLogRecordFactory() for more information about the how the

factory is called.

logging.debug(msg, *args, **kwargs)

This is a convenience function that calls Logger.debug(), on the root

logger. The handling of the arguments is in every way identical

to what is described in that method.

The only difference is that if the root logger has no handlers, then

basicConfig() is called, prior to calling debug on the root logger.

For very short scripts or quick demonstrations of logging facilities,

debug and the other module-level functions may be convenient. However,

most programs will want to carefully and explicitly control the logging

configuration, and should therefore prefer creating a module-level logger and

calling Logger.debug() (or other level-specific methods) on it, as

described at the beginnning of this documentation.

logging.info(msg, *args, **kwargs)

Logs a message with level INFO on the root logger. The arguments and behavior

are otherwise the same as for debug().

logging.warning(msg, *args, **kwargs)

Logs a message with level WARNING on the root logger. The arguments and behavior

are otherwise the same as for debug().

Note

There is an obsolete function warn which is functionally

identical to warning. As warn is deprecated, please do not use

it - use warning instead.

logging.error(msg, *args, **kwargs)

Logs a message with level ERROR on the root logger. The arguments and behavior

are otherwise the same as for debug().

logging.critical(msg, *args, **kwargs)

Logs a message with level CRITICAL on the root logger. The arguments and behavior

are otherwise the same as for debug().

logging.exception(msg, *args, **kwargs)

Logs a message with level ERROR on the root logger. The arguments and behavior

are otherwise the same as for debug(). Exception info is added to the logging

message. This function should only be called from an exception handler.

logging.log(level, msg, *args, **kwargs)

Logs a message with level level on the root logger. The arguments and behavior are otherwise the same as for debug().

logging.disable(level=CRITICAL)

Provides an overriding level level for all loggers which takes precedence over the logger·s own level. When the need arises to temporarily throttle logging output down across the whole application, this function can be useful. Its effect is to disable all logging calls of severity level and below, so that if you call it with a value of INFO, then all INFO and DEBUG events would be discarded, whereas those of severity WARNING and above would be processed according to the logger·s effective level. If logging.disable(logging.NOTSET) is called, it effectively removes this overriding level, so that logging output again depends on the effective levels of individual loggers.

Note that if you have defined any custom logging level higher than CRITICAL (this is not recommended), you won·t be able to rely on the default value for the level parameter, but will have to explicitly supply a suitable value.

Changed in version 3.7: The level parameter was defaulted to level CRITICAL. See bpo-28524 for more information about this change.

logging.addLevelName(level, levelName)

Associates level level with text levelName in an internal dictionary, which is

used to map numeric levels to a textual representation, for example when a

Formatter formats a message. This function can also be used to define

your own levels. The only constraints are that all levels used must be

registered using this function, levels should be positive integers and they

should increase in increasing order of severity.

Note

If you are thinking of defining your own levels, please see the

section on Custom Levels.

logging.getLevelNamesMapping()

Returns a mapping from level names to their corresponding logging levels. For example, the

string ·CRITICAL· maps to CRITICAL. The returned mapping is copied from an internal

mapping on each call to this function.

Added in version 3.11.

logging.getLevelName(level)

Returns the textual or numeric representation of logging level level.

If level is one of the predefined levels CRITICAL, ERROR,

WARNING, INFO or DEBUG then you get the

corresponding string. If you have associated levels with names using

addLevelName() then the name you have associated with level is

returned. If a numeric value corresponding to one of the defined levels is

passed in, the corresponding string representation is returned.

The level parameter also accepts a string representation of the level such

as ·INFO·. In such cases, this functions returns the corresponding numeric

value of the level.

If no matching numeric or string value is passed in, the string

·Level %s· % level is returned.

Note

Levels are internally integers (as they need to be compared in the

logging logic). This function is used to convert between an integer level

and the level name displayed in the formatted log output by means of the

%(levelname)s format specifier (see LogRecord attributes), and

vice versa.

Changed in version 3.4: In Python versions earlier than 3.4, this function could also be passed a

text level, and would return the corresponding numeric value of the level.

This undocumented behaviour was considered a mistake, and was removed in

Python 3.4, but reinstated in 3.4.2 due to retain backward compatibility.

logging.getHandlerByName(name)

Returns a handler with the specified name, or None if there is no handler

with that name.

Added in version 3.12.

logging.getHandlerNames()

Returns an immutable set of all known handler names.

Added in version 3.12.

logging.makeLogRecord(attrdict)

Creates and returns a new LogRecord instance whose attributes are

defined by attrdict. This function is useful for taking a pickled

LogRecord attribute dictionary, sent over a socket, and reconstituting

it as a LogRecord instance at the receiving end.

logging.basicConfig(**kwargs)

Does basic configuration for the logging system by creating a

StreamHandler with a default Formatter and adding it to the

root logger. The functions debug(), info(), warning(),

error() and critical() will call basicConfig() automatically

if no handlers are defined for the root logger.

This function does nothing if the root logger already has handlers

configured, unless the keyword argument force is set to True.

Note

This function should be called from the main thread

before other threads are started. In versions of Python prior to

2.7.1 and 3.2, if this function is called from multiple threads,

it is possible (in rare circumstances) that a handler will be added

to the root logger more than once, leading to unexpected results

such as messages being duplicated in the log.

The following keyword arguments are supported.

Format

Description

filename

Specifies that a FileHandler be

created, using the specified filename,

rather than a StreamHandler.

filemode

If filename is specified, open the file

in this mode. Defaults

to 'a'.

format

Use the specified format string for the

handler. Defaults to attributes

levelname, name and message

separated by colons.

datefmt

Use the specified date/time format, as

accepted by time.strftime().

style

If format is specified, use this style

for the format string. One of '%',

'{' or '$' for printf-style,

str.format() or

string.Template respectively.

Defaults to '%'.

level

Set the root logger level to the specified

level.

stream

Use the specified stream to initialize the

StreamHandler. Note that this

argument is incompatible with filename -

if both are present, a ValueError is

raised.

handlers

If specified, this should be an iterable of

already created handlers to add to the root

logger. Any handlers which don·t already

have a formatter set will be assigned the

default formatter created in this function.

Note that this argument is incompatible

with filename or stream - if both

are present, a ValueError is raised.

force

If this keyword argument is specified as

true, any existing handlers attached to the

root logger are removed and closed, before

carrying out the configuration as specified

by the other arguments.

encoding

If this keyword argument is specified along

with filename, its value is used when the

FileHandler is created, and thus

used when opening the output file.

errors

If this keyword argument is specified along

with filename, its value is used when the

FileHandler is created, and thus

used when opening the output file. If not

specified, the value ·backslashreplace· is

used. Note that if None is specified,

it will be passed as such to open(),

which means that it will be treated the

same as passing ·errors·.

Changed in version 3.2: The style argument was added.

Changed in version 3.3: The handlers argument was added. Additional checks were added to

catch situations where incompatible arguments are specified (e.g.

handlers together with stream or filename, or stream

together with filename).

Changed in version 3.8: The force argument was added.

Changed in version 3.9: The encoding and errors arguments were added.

logging.shutdown()

Informs the logging system to perform an orderly shutdown by flushing and

closing all handlers. This should be called at application exit and no

further use of the logging system should be made after this call.

When the logging module is imported, it registers this function as an exit

handler (see atexit), so normally there·s no need to do that

manually.

logging.setLoggerClass(klass)

Tells the logging system to use the class klass when instantiating a logger.

The class should define __init__() such that only a name argument is

required, and the __init__() should call Logger.__init__(). This

function is typically called before any loggers are instantiated by applications

which need to use custom logger behavior. After this call, as at any other

time, do not instantiate loggers directly using the subclass: continue to use

the logging.getLogger() API to get your loggers.

logging.setLogRecordFactory(factory)

Set a callable which is used to create a LogRecord.

Parameters:

factory · The factory callable to be used to instantiate a log record.

Added in version 3.2: This function has been provided, along with getLogRecordFactory(),
to

allow developers more control over how the LogRecord representing

a logging event is constructed.

The factory has the following signature:

factory(name, level, fn, lno, msg, args, exc_info, func=None, sinfo=None, **kwargs)

name:

The logger name.

level:

The logging level (numeric).

fn:

The full pathname of the file where the logging call was made.

lno:

The line number in the file where the logging call was made.

msg:

The logging message.

args:

The arguments for the logging message.

exc_info:

An exception tuple, or None.

func:

The name of the function or method which invoked the logging

call.

sinfo:

A stack traceback such as is provided by

traceback.print_stack(), showing the call hierarchy.

kwargs:

Additional keyword arguments.

Module-Level Attributes

logging.lastResort

A ·handler of last resort· is available through this attribute. This

is a StreamHandler writing to sys.stderr with a level of

WARNING, and is used to handle logging events in the absence of any

logging configuration. The end result is to just print the message to

sys.stderr. This replaces the earlier error message saying that

·no handlers could be found for logger XYZ·. If you need the earlier

behaviour for some reason, lastResort can be set to None.

Added in version 3.2.

logging.raiseExceptions

Used to see if exceptions during handling should be propagated.

Default: True.

If raiseExceptions is False,

exceptions get silently ignored. This is what is mostly wanted

for a logging system - most users will not care about errors in

the logging system, they are more interested in application errors.

Integration with the warnings module

The captureWarnings() function can be used to integrate logging

with the warnings module.

logging.captureWarnings(capture)

This function is used to turn the capture of warnings by logging on and

off.

If capture is True, warnings issued by the warnings module will

be redirected to the logging system. Specifically, a warning will be

formatted using warnings.formatwarning() and the resulting string

logged to a logger named 'py.warnings' with a severity of WARNING.

If capture is False, the redirection of warnings to the logging system

will stop, and warnings will be redirected to their original destinations

(i.e. those in effect before captureWarnings(True) was called).

See also

Module logging.config

Configuration API for the logging module.

Module logging.handlers

Useful handlers included with the logging module.

PEP 282 - A Logging System

The proposal which described this feature for inclusion in the Python standard

library.

Original Python logging package

This is the original source for the logging package. The version of the

package available from this site is suitable for use with Python 1.5.2, 2.1.x

and 2.2.x, which do not include the logging package in the standard

library.

© Copyright 2001-2026, Python Software Foundation.

This page is licensed under the Python Software Foundation License Version 2.

Examples, recipes, and other code in the documentation are additionally licensed under the Zero Clause BSD License.

See History and License for more information.

The Python Software Foundation is a non-profit corporation.

Please donate.

Last updated on Oct 26, 2026 (15:00 UTC).

Found a bug?

Created using Sphinx 8.2.3.